

Predicción de churn: servicio MLOps end-to-end

Informe técnico del proyecto

- Agustin Haramboure
- Diego Pérez
- Estefania Olivares

URL: https://github.com/Agustin-uai-MDS/MLOps_churn_tarea.git

1. Problema y datos

El servicio resuelve un problema de clasificación binaria: predecir si un cliente de una compañía de telecomunicaciones va a darse de baja (churn), a partir de sus datos de cuenta y servicios contratados. La predicción no se entrega solo como etiqueta, sino como una probabilidad junto con un umbral de decisión configurable, lo que permite ajustar la sensibilidad del sistema según el costo de negocio de cada tipo de error.

Los datos provienen del dataset público Telco Customer Churn (Kaggle / IBM), con 7.043 clientes y 21 columnas que cubren demografía, tipo de contrato, servicios contratados y cobros. Es un dataset ficticio, sin información personal identificable real, lo que cumple la condición de acceso legítimo exigida por la pauta.

La exploración inicial (nulos, outliers, variables categóricas) se documentó en un notebook de análisis. El feature engineering que efectivamente usa el servicio en producción está implementado por separado en el módulo de entrenamiento, para que la API no dependa de código de notebook.

2. Modelo y resultados

Se entrenó un RandomForestClassifier de scikit-learn con semilla fija (42) y una partición estratificada 80/20 entre entrenamiento y prueba, de forma que la proporción de clientes con y sin churn se mantiene similar en ambos conjuntos. El script de entrenamiento es reproducible y ejecutable de forma independiente del servicio de inferencia.

Las métricas se calculan sobre el conjunto de prueba (1.409 clientes que el modelo nunca vio durante el entrenamiento):

Accuracy	Precision	Recall	F1	ROC-AUC
0.7658	0.5412	0.7727	0.6366	0.8428

El target está desbalanceado (26,5% de churn), por lo que un modelo que siempre predijera "no churn" ya alcanzaría un 73,5% de accuracy sin haber aprendido nada. Por eso la métrica relevante es el ROC-AUC (0,84), que sí refleja la capacidad del modelo de separar ambas clases. Este valor es consistente con el techo reportado por la comunidad para este dataset con este mismo conjunto de columnas: no es una limitación del pipeline, sino información que el dataset no contiene (ver sección 4).

Cada entrenamiento produce tres artefactos versionados: el modelo (model.joblib), el preprocesador (preprocessor.joblib) y un archivo de metadatos (metadata.json) con las features usadas, la versión de scikit-learn, la fecha de entrenamiento, las métricas y los rangos de entrenamiento (percentiles 1-99) de cada

variable numérica. Estos rangos se calculan solo sobre el set de entrenamiento, nunca sobre el de prueba, para no filtrar información del set de evaluación al artefacto.

3. Decisiones de diseño

Separación entrenamiento / inferencia

El entrenamiento (training/) y el servicio (app/) están completamente desacoplados: la API carga los artefactos ya serializados y nunca importa código de entrenamiento en tiempo de ejecución. Esto permite reentrenar el modelo sin tocar el servicio, y viceversa.

Contrato de API (FastAPI)

El modelo se carga una sola vez al arrancar el servicio, no en cada request. La API expone GET /health (estado del servicio y del modelo), POST /predict y POST /predict/batch, GET /model/schema (features esperadas, métricas y rangos de entrenamiento del modelo cargado) y documentación interactiva automática en /docs. Las entradas y salidas están validadas con modelos Pydantic, de forma que una entrada inválida devuelve un error con código HTTP correcto y un mensaje que indica qué campo falló, en vez de un 500 genérico.

Contenerización

El servicio se empaqueta en Docker con una imagen base slim, sin incluir las dependencias de entrenamiento que no se necesitan en producción. docker compose up levanta el sistema completo con un solo comando, sin pasos manuales adicionales, quedando disponible en <http://localhost:8000>.

CI/CD (GitHub Actions)

El pipeline corre en cada push y pull_request sobre main, con jobs encadenados: lint → test → build → smoke, cada uno condicionado a que el anterior haya pasado. El job de smoke test es el más relevante: levanta el contenedor ya construido y le hace curl real a /health y /predict, verificando el artefacto final y no el código fuente. Al crear un tag v*, se agrega un quinto job (publish) que sube esa misma imagen ya probada a GitHub Container Registry.

Pruebas y calidad

La suite de pytest cubre el contrato de la API, la validación de entradas, casos bordes y errores esperados, y corre sin red ni credenciales. El linting y formateo se maneja con ruff, configurado en el repositorio.

4. Funcionalidades adicionales implementadas

Más allá de los requisitos obligatorios, el servicio incorpora dos capacidades que apuntan a los puntos bonus de la pauta:

- Validación de rango de entrada (drift simple): cada predicción informa en out_of_range_features si alguna variable numérica del request cae fuera del percentil 1-99 visto en entrenamiento, como señal de que la predicción podría ser menos confiable.
- Métricas de servicio: GET /metrics expone conteo de requests, errores y latencia promedio por endpoint.

Ambas quedan documentadas con ejemplos reales de respuesta en el README del repositorio.

5. Limitaciones conocidas y trabajo futuro

- Techo del dataset: con las 21 columnas disponibles (sin historial de reclamos, ofertas de competencia ni satisfacción del cliente), un ROC-AUC entre 0,84 y 0,86 es el rango consistentemente reportado por la comunidad para este dataset. No es atribuible al pipeline.
- Sin registro de modelos: los artefactos se versionan directamente en Git, no en un model registry como MLflow.
- Métricas no distribuidas: GET /metrics acumula el conteo en el proceso de un solo worker de uvicorn; con más de un worker o réplica cada uno lleva su propio conteo, sin un total agregado real.
- Tamaño de imagen (~760 MB): esperable para un stack con scikit-learn, pandas, numpy y scipy, ya evitando dependencias de entrenamiento extra.

Con más tiempo, el equipo incorporaría un registro de modelos con MLflow en lugar de artefactos versionados en Git, y probaría HistGradientBoostingClassifier para intentar mejorar el ROC-AUC.

6. Quién hizo qué

Integrante	Módulos / responsabilidad
Diego Perez	Data , notebooks , Training ,Test y requerimientos
Agustin Haramboure	Credenciales, Fast API y CI/CD
Estefania Olivares	Models , Docker y Readme

7. Uso de asistentes de IA

Se usó Claude (Anthropic, vía Claude Code) como asistente durante el desarrollo: Arquitectura de la API (separación entre configuración, contratos, predicción y rutas) y diseño y depuración del pipeline de CI/CD, incluyendo pruebas locales del workflow antes de cada push. Cada decisión de diseño fue discutida y revisada por el equipo antes de aplicarse.